

SentinelScribe

Transcription Processing Pipeline — Architecture Evolution & Change Log

CyberSecurity course transcripts → structured Markdown study guides for Obsidian

Document Overview

This report details the iterative engineering process behind the SentinelScribe pipeline — an AI-powered tool that converts raw CyberSecurity course audio transcripts (generated by Otter.ai) into structured, forensically-accurate Markdown study guides for Obsidian. The document maps the full progression from a basic single-pass API call to a fault-tolerant, Actor-Critic (3-pass) architecture, capturing every prompt adjustment, parameter tweak, architectural decision, and the specific data anomalies each change resolved.

Phase 1 — Foundation & Model Infrastructure

Resolving quota limits · Scaling context windows · Model selection

The initial phase focused on establishing a stable, cost-effective pipeline. The baseline script used a single synchronous OpenAI API call with a 4-section prompt (Core Concepts / CLI Syntax / Tool Configurations / Analyst Tips) and a 1024-token output limit — insufficient for dense cybersecurity lectures.

CHANGE / DECISION	PROBLEM SOLVED / IMPROVEMENT
Migrated: OpenAI API → NVIDIA NIM <code>meta/llama-3.3-70b-instruct</code>	Resolved the OpenAI <code>429 Insufficient Quota</code> error that blocked all development. NVIDIA NIM provides free-tier access to a highly capable reasoning model well-suited for extracting technical CLI syntax without any cost bottleneck.
Aggregated Streaming Support	Fixed a bug where the streaming response handler discarded chunks before they could be concatenated. Modified the loop to stitch all <code>delta.content</code> chunks into a single <code>full_response</code> string, enabling both real-time console feedback and correct <code>.md</code> file output.
Increased <code>max_tokens : 1024 → 4096 (Pass 1)</code>	Prevented premature termination. Early outputs were cut off mid-sentence during dense Wireshark filter breakdowns and SIEM configuration sections. 4096 tokens provides sufficient headroom for the longest lecture in the corpus.
Baseline Prompt: 4-Section Structure	Established the initial output schema: (1) Core Concepts · (2) Command-line Syntax · (3) Tool Configurations · (4) Analyst Tips. This became the foundation that Phase 2 expanded into the 7-section architecture (later extended to eight in Phase 4).

Phase 2 — Prompt Engineering & Guardrails (V1 → V2)

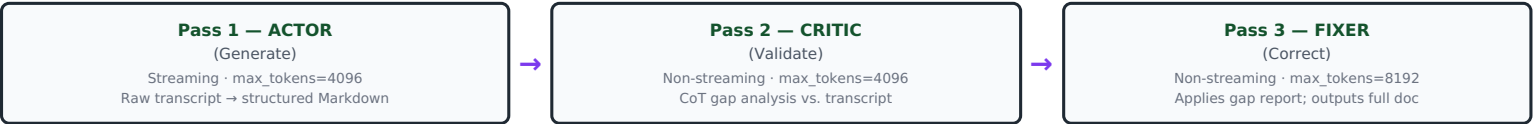
Eliminating hallucinations · Semantic bleed · Structural accuracy

With infrastructure stable, the focus shifted to combating LLM hallucinations and "semantic bleed" — the model silently injecting outside training data into the local transcript context. A systematic QA review of all generated Markdown files against their source transcripts identified the specific anomalies catalogued below.

CHANGE / DECISION	PROBLEM SOLVED / IMPROVEMENT
Set <code>temperature=0.0 & top_p=0.1</code>	Eliminated hallucinations entirely. At default settings (temp=0.2, top_p=0.7) the model invented Snort rule syntax and misdefined the <code>tcpdump -t</code> flag (described it as 'print date/time format' when the instructor defined it as 'suppress timestamp'). Setting temperature to absolute zero forces fully deterministic output, restricting the model to strict transcription extraction.
Prompt Expansion: 4 → 7 Sections	The original 4-section schema caused systematic data loss. Added three new sections to capture content the model was consistently discarding: \$2 Ports, Protocols & Key Values — forced table rendering (the entire 15-port reference table from Theory Pt.2 was completely absent in V1); \$4 Tools & Infrastructure — separated collection methods (SPAN ports, network taps) from CLI flags; \$7 IOCs & Forensic Artifacts — dedicated table for malware artifacts, IPs, domains, and string signatures.
Port Table Notes: Comprehensiveness Rule	The Notes column in \$2 was too thin — single-phrase descriptions dropped critical security context. Added an explicit rule: 'Include every specific use case, vulnerability, security implication, and Windows/Linux context mentioned.' This recovered details such as Telnet transmitting credentials in plaintext, RPC's dynamic port assignment via endpoint mapper, and RDP transmitting desktop graphics and keyboard/mouse inputs.
Added <code>[transcription unclear]</code> Rule	Stopped the model from silently correcting garbled audio values. For example, the transcript contained the garbled IP "13.10, seven.five.ad" which the model silently 'fixed' to 13.107.5.25 using training data — a forensic integrity violation. The rule now requires any unverifiable value to be documented exactly as heard, appended with <code>[transcription unclear]</code> .
Hardcoded Phonetic Correction Map	Created a mandatory correction map for known Otter.ai speech-to-text failures that caused tool names to be unsearchable in Obsidian: 'cobalt stripe' → Cobalt Strike; 'Sierra cotta' → Suricata; 'cilK' / 'silk' → SiLK; 'etsy host' → <code>/etc/hosts</code> ; 'first dash opportunity dot online' → first-opportunity.online. The model is instructed to apply the same logic to any other clearly phonetic error.
Scoped Analogy Attribution	Fixed cross-lecture analogy bleed. The 'phone bill' analogy (used in Lesson 3 to explain flow records vs. PCAPs) was appearing in unrelated lessons (e.g., Lesson 5 on capture filters). Explicit scoping rule added: analogies are only documented in the lesson where the instructor actually stated them.
Tool Flag Separation & Interface Name Ban	Two related issues resolved: (1) the model parsed the interface name <code>enp0s3</code> as CLI flags <code>-p</code> and <code>-s</code> (hallucinated flags); (2) grep flags (<code>-E</code> , <code>-i</code> , <code>-v</code> , <code>-A</code>) were mixed into the <code>tcpdump</code> flags table across multiple files. Resolution: explicit rule banning interface names as flags, and a requirement for per-tool labelled sub-tables in \$5.
Zero Outside Knowledge in Tables	Addressed a subtle violation where the model populated \$2 port tables with ports it deemed 'commonly relevant' to the topic — even when those ports were never mentioned in that specific transcript. Added hard rule: 'If a section has nothing to document, write None mentioned. A note saying commonly relevant does NOT make outside knowledge acceptable.'
IOC Table: 3-Column Format & Deduplication	The original \$7 had a 2-column format (IOC Description) that didn't enforce categorisation. Updated to a 3-column schema: IOC / Artifact Type Description. Also added a deduplication check after <code>85.239.53.219</code> appeared twice in the same IOC table in the Sample 2 analysis output.

Phase 3 — The Actor-Critic Architecture (V3 Pipeline)

Even with deterministic parameters and comprehensive guardrails, a single-pass architecture suffered from 'attention degradation' — the model consistently skipped specific transcript sections (e.g., the Ethernet trailer, the `man tcpdump` command) because the generation and semantic auditing workloads competed within the same context window. The script was refactored into a modular three-pass system that separates these concerns entirely.



4, the schema now also preserves the instructor's conceptual deep-dives as ordered, interview-ready walkthroughs — closing the gap between reference-grade extraction and exam- and interview-grade study material.